HR

INTERNAL PROJECT REPORT

AIRA HRMS

A multi-tenant Human Resource Management System — architecture, current development phase, quality & security posture, and the forward feature roadmap.

REPORT DATE

23 September 2026

PREPARED BY

Engineering

STATUS

MVP — active development

STACK

Next.js 16 · Postgres

DEPLOYMENT

Not yet deployed (dev)

TENANCY MODEL

Single app, isolated tenants

1. Executive summary

AIRA HRMS is a multi-tenant HR SaaS application: multiple companies ("tenants") share one deployed application while their data stays strictly isolated from one another. Every tenant-owned record carries a `tenant_id` that the server derives from the authenticated user's membership — never from client input — and this isolation is covered by an automated end-to-end test suite.

The system currently ships two full experiences from a single codebase: a **full HR console** for owners, admins, HR, and managers (employee records, attendance, leave, payroll, performance, company settings, audit log), and a **self-service employee portal** (attendance check-in/out, leave requests and planning, payslips, documents, notifications). A separate, lightweight **platform super-admin** view gives cross-tenant visibility for operating the platform itself.

62	~30	44	6
API ROUTE HANDLERS	DATABASE TABLES	AUTOMATED E2E TESTS, ALL PASSING	ROLES + PLATFORM SUPER ADMIN

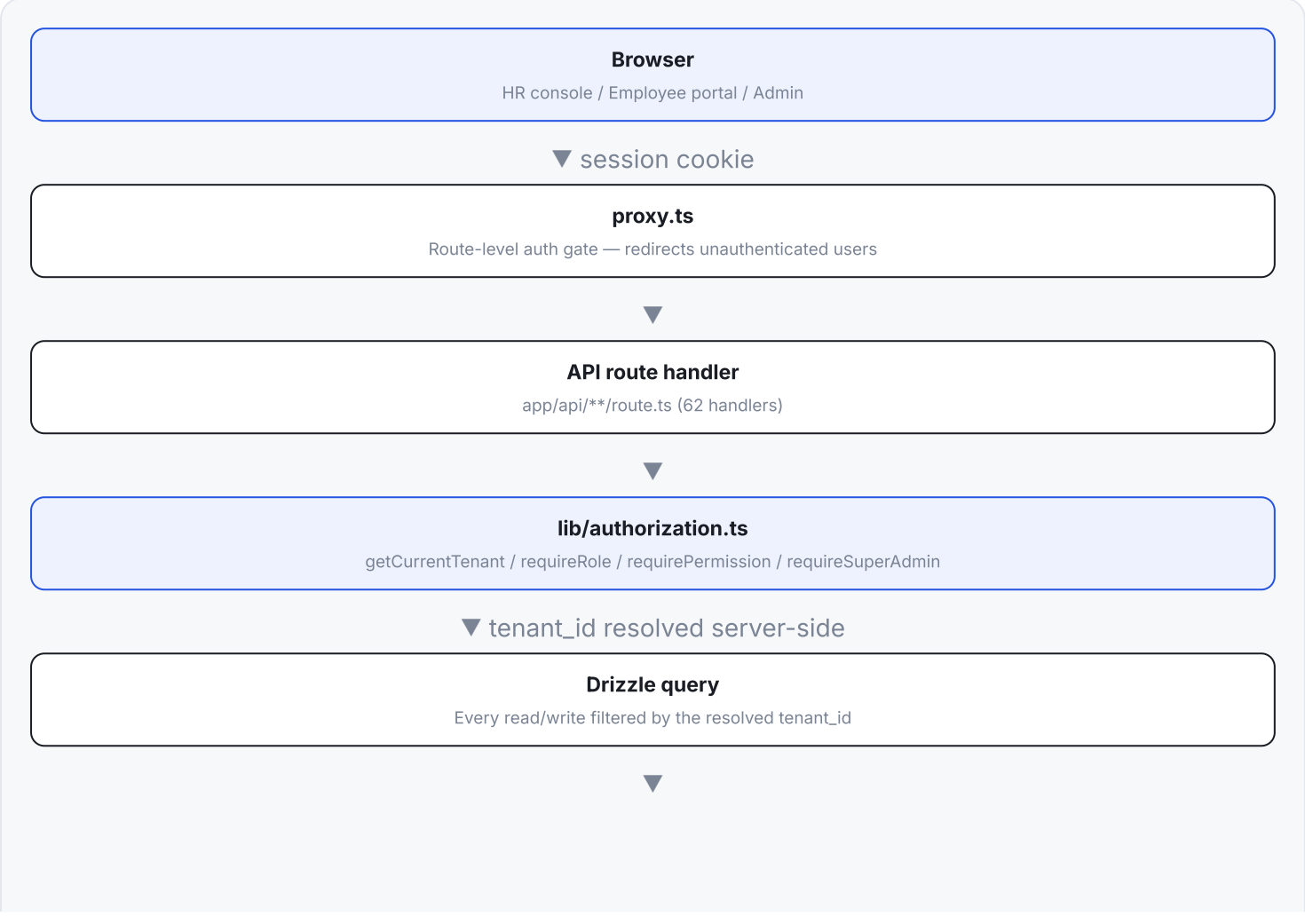
This report covers, in order: the system architecture, what has been built and verified to date, the quality/security work carried out in the most recent development pass, known limitations, and the proposed feature roadmap for the next phases of work.

2. System architecture

2.1 Technology stack

LAYER	CHOICE	NOTES
Framework	Next.js 16 (App Router, Turbopack)	Single codebase serves pages and API routes
Language	TypeScript	End to end, including schema and validation
UI	Tailwind CSS v4 + shadcn/ui (base-nova, @base-ui/react)	Themeable design-token system (see §2.4)
Auth	Better Auth (email/password)	Session/cookie based, Drizzle-backed
ORM	Drizzle ORM + drizzle-kit	Schema-as-code, generated SQL migrations
Database	Neon Postgres (serverless, pooled)	Indexed on every tenant/employee/status lookup column
Validation	Zod	Runtime request validation on write endpoints
Testing	Playwright (e2e)	Runs against a real seeded database

2.2 Request & authorization flow



Neon Postgres

Indexed, tenant-scoped tables

Four authorization helpers, one source of truth: `getCurrentTenant()` resolves session → user → membership → tenant; `requireRole()` additionally checks the caller's role against an allow-list; `requirePermission()` checks a specific permission via `lib/permissions.ts`; `requireSuperAdmin()` is the separate, tenant-independent check gating the platform admin console.

2.3 Codebase layout

PATH	PURPOSE
`app/(hr)/`	HR console — dashboard, employees, attendance, leave, payroll, performance, organization, reports, settings
`app/employee/`	Employee self-service portal — own attendance, leave, payslips, documents, notifications
`app/admin/`	Platform super-admin console (cross-tenant)
`app/api/`	62 route handlers backing every screen above
`components/hr/`	HR console building blocks (sidebar, page header, data tables, status badges)
`components/ui/`	shadcn/ui primitives (buttons, cards, dialogs, dropdowns, tables…)
`db/schema/`	Drizzle table definitions, one module per domain
`lib/`	Auth config, authorization guards, RBAC source of truth, notification/audit helpers
`e2e/`	Playwright end-to-end test suite

2.4 Multi-tenancy model

Every tenant-owned table (`employees`, `attendance`, `leave_requests`, `payroll_runs`, and roughly two dozen others) carries a `tenant_id` foreign key. No query parameter, header, or request body field is ever treated as a tenant identifier — the tenant is always resolved server-side from the session. This is enforced consistently enough that it is covered by dedicated automated tests (§4.1) rather than relying on manual review alone.

> **Roles.** Per-tenant membership role is one of `owner`, `admin`, `hr`, `manager`, `employee`, `member`. Platform Super Admin is a separate concept — a boolean flag on the user, not a membership role — giving cross-tenant operational access independent of any single tenant.

3. Current development phase — feature inventory

The project is a functioning MVP: every module below is implemented, backed by real API routes and database tables, and reachable in the running application. Nothing in this section is a mock-up.

3.1 HR console (owner / admin / hr / manager)

MODULE	CAPABILITY
Dashboard	Tenant-wide stats — headcount, attendance today, pending leave, payroll summary, departments
Employees	Directory, search/filter, create/edit, per-employee profile with documents
Organization	Company profile, departments, designations, locations, reporting structure
Attendance	Daily records, per-employee history, manual correction
Leave	Requests (approve/reject), balances, leave types & rules, encashment, adjustment, LTC claims
Payroll	Salary structures & components, payroll runs, payslip generation
Performance	Goals and review cycles per employee
Reports	Cross-domain tenant reporting (feeds the dashboard)
Settings	Company details, users & memberships, roles & permissions viewer, system preferences, audit log

3.2 Employee self-service portal

MODULE	CAPABILITY
Dashboard	Today's attendance with check-in/check-out, quick actions, at-a-glance counts
Attendance	Personal attendance history
Leave	Balances, apply/cancel requests, personal leave planner, encashment, adjustment requests, LTC claims, leave rules reference
Payslips	Own payslip history
Documents	Personal document library
Notifications	In-app notifications with unread tracking

3.3 Platform administration

A separate super-admin console (`/admin`) gives visibility across every tenant on the platform, gated independently of the per-tenant role system via `users.isSuperAdmin` .

4. Quality, security & performance

4.1 Automated test coverage

The full Playwright suite currently stands at **44 end-to-end tests, all passing**, run against a real seeded database. It is organized into three areas:

- **Auth & route protection** — form rendering/validation, and that unauthenticated users are redirected away from every protected route while receiving a clean `401` (not a crash) from protected APIs.
- **Tenant isolation & RBAC** — login redirects by role, cross-tenant data isolation between the two demo tenants, and a dedicated set of broken-access-control / IDOR checks (an employee cannot approve their own leave, correct their own attendance, view a colleague's data, or reach tenant-wide payroll/payslip data).
- **Feature regression** — the full employee self-service surface (every section loads), leave planner/encashment/adjustment/LTC balance math, and leave-request cancellation rules.

4.2 Security review findings (this cycle)

A focused authorization/data-isolation audit of all 62 API routes was carried out this cycle. Two real issues were found and fixed, both now covered by regression tests:

ISSUE	IMPACT	RESOLUTION
Cross-employee data leak `GET /api/performance/goals`	Any authenticated employee (lowest privilege) could read every other employee's performance goals in the tenant, not just their own.	Query now scoped to the caller's own `employeeId` unless their role is manager-tier or above. Covered by a new e2e assertion.
Inconsistent role gate `GET /api/company`	Managers were blocked from viewing the Company Profile page, while every comparable read endpoint (departments, designations, locations, employees) correctly allowed the manager role — a functional bug, not a leak.	Aligned the allowed-role list with the rest of the codebase's established convention for read access.

> **Outcome:** no other route audited showed a missing role check, missing tenant scoping, or IDOR-style lookup-by-id-only pattern.

4.3 Performance

A latency pass added indexes on every column actually used to filter tenant/employee/status data (49 indexes across the schema — previously only the auth tables were indexed), removed duplicate authorization round-trips present in several routes, and parallelized independent database calls within request handlers using `Promise.all` where they had previously run sequentially.

4.4 UI / UX

Both the HR console and the employee portal received a full visual pass: a consistent design-token system (brand color, spacing, radius), a redesigned navigation shell (workspace switcher, notification indicators, user menu), a professional split-screen sign-in/sign-up flow, and a restyled employee dashboard. A production-mode fix also corrected a font-loading regression that had silently fallen back to the browser default serif typeface across the whole application.

5. Known limitations

AREA	LIMITATION
Payroll	MVP-level only. Computes gross/net pay from basic salary, allowances and deductions — no statutory compliance (PF, ESI, TDS, or equivalent) for any jurisdiction. Not production-ready for real payroll processing.
Notifications	Fire only for leave events, payroll run generation, and membership role changes. Attendance, performance, and organization changes don't yet raise notifications.
Deployment	No production deployment yet — the application has only been run in local/development mode to date.
Data residency / latency	Database is hosted in a single Neon region; running the app far from that region adds real network latency to every request. Not yet addressed by a production deployment topology.

6. Feature upgrade plan

The following phases were scoped during planning but not yet built. They are sequenced by dependency and value, reusing the app's existing conventions (Drizzle schema-per-domain, `requireRole` / `getCurrentTenant` guards, the submit → notify-manager → admin-approves pattern already proven by leave requests).

Phase A — Time & Attendance self-service

`Planned`

Employee-initiated shift-change requests, short time-off requests, and attendance regularization ("swipe correction") requests, each with an HR approval workflow mirroring the existing leave-request pattern. Includes a tenant-configurable holiday calendar with optional-holiday selection.

Phase B — Performance management depth

`Planned`

Self-assessment on review cycles, a personal achievements log, and the missing admin-side rate/finalize workflow for review cycles (the underlying API already exists; only the UI is missing). An "Appraisal Sheet" report composing goals, achievements and reviews for one employee.

Phase C — Employee profile, salary & tax self-service

`Planned`

Personal/bank/statutory detail management, qualifications, skills and emergency-contact records; a proper link between salary structures and their components so employees can view a CTC breakdown and download a printable CTC letter and payslip; tax declaration submission with HR review, and a net-taxable-income projection summary.

Phase D — Production readiness

`Planned`

- First production deployment (Vercel + Neon), with the database region chosen to minimize latency to the deployed app.
- Statutory payroll compliance for the target jurisdiction, if the product is to be used for real payroll.
- Expand in-app notifications to attendance, performance, and organization-change events.

> **Sequencing note.** Phases A–C are independent of each other and can be reprioritized freely; Phase D (production readiness) is recommended before any phase is used with real company data, since payroll compliance and deployment topology affect all of them.